Computing Approximate Derivatives

Sometimes gradient and/or Hessian information is unavailable even for smooth functions. For example, we may not have closed-form expressions or $f(x)$ is obtained through blackbox code. We can approximate derivatives using finite differences — with care!

Suppose we wish to determine $\nabla f(x)$ for a particular point x. We approximate each component by considering the limit definition of a derivative:

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x+h\hat{x}_i) - f(x)}{h}$$

We can say that, for some small enough h,

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x+h\hat{x}_i) - f(x)}{h}$$

one extra function evaluation at $x+h\hat{x}_i$. This is similar in cost to the direct calc.

In an optimization setting we do not want to spend a lot of time deciding a good value for h — this would involve many extra function evaluations. Instead, we can determine a likely good h just by considering the properties of the numerical calculation.

Consider the numerical task of estimating the derivative of $f(x) = \sin(x)$ at $x=1$, using limited precision arithmetic — say 9 digit precision. We have the function value

$$f(1) = \sin(1) = 0.841470984 \quad \text{(9 digits!)},$$

and our estimate strategy for $f'(x)$:

$$g(x) = \frac{\sin(x+h) - \sin(x)}{h}$$

If we had infinite precision arithmetic, then we could choose h as small as we desire because $\lim_{h \to 0} g(x) = f'(x)$.

The derivative value we seek is (unknown to us!):

$$f'(1) = \cos(1) = 0.540302305 \quad (9 \text{ digits!}).$$

Consider the results of trying various values of h:

h	$\sin(1+h)$	$g(1)$
10^{-9}	0.841470985	1.0
10^{-8}	0.841470990	0.6211
10^{-7}	0.841471038	0.54839
10^{-6}	0.841471525	0.541108
10^{-5}	0.841476387	0.5403786
10^{-4}	0.841525010	0.54026832
10^{-3}	0.842010866	0.539882289
10^{-2}	0.846831844	0.5360860617
10^{-1}	0.891207360	0.4973637600
1	0.909297426	0.0678264420

when h is too small we do not have sufficient precision for a meaningful computation

when h is too large we are finding the slope of a secant line that is not local to x.

We choose to keep half of the precision as a compromise:

$$h = \sqrt{10^{-9}} \approx 3 \times 10^{-5}$$

The result with this h is $g(1) = 0.540314413$.

For double-precision computing $eps = 2^{-52}$ so $h = 2^{-26} \approx 1.6 \times 10^{-8}$.
This analysis is appropriate for problems with $|x| \sim 1$ and $x + h \sim x$.

We might also consider a central differencing scheme in an attempt to improve accuracy. We have:

$$\frac{\partial f}{\partial x_i}(x) = \frac{1}{2}\left(\lim_{h\to 0} \frac{f(x+h\hat{x}_i)-f(x)}{h} + \lim_{h\to 0} \frac{f(x)-f(x+h\hat{x}_i)}{h} \right)$$

$$= \frac{1}{2} \lim_{h\to 0} \frac{f(x+h\hat{x}_i)-f(x-h\hat{x}_i)}{h}$$

$$\approx \frac{f(x+h\hat{x}_i)-f(x-h\hat{x}_i)}{2h}$$

This method requires $2n$ extra function evaluations, but has the benefit of increased accuracy. This approach is usually avoided in an iterative optimization algorithm context in which iterates need only be very approximate solutions to a line search.

Suppose we wanted to estimate Hessian information.

$$[\nabla^2 f]_{ij} = \frac{\partial}{\partial x_i}\frac{\partial f}{\partial x_j}(x) = \frac{\partial}{\partial x_j}[\nabla f(x)]_i$$

$$\approx \frac{[\nabla f(x+k\hat{x}_j)]_i - [\nabla f(x)]_i}{k}$$

symmetry reduces the total number of extra gradient evaluations to $\frac{1}{2}n(n+1) < n^2$ and $k = \sqrt{eps}$ as before.

The above method is appropriate if $\nabla f(x)$ is computed to full precision. If $\nabla f(x)$ is not available to full precision we have

$$[\nabla^2 f]_{ij} = \frac{\partial}{\partial x_i}\frac{\partial f}{\partial x_j}(x) \approx \frac{\partial}{\partial x_j} \frac{f(x+h\hat{x}_i)-f(x)}{h}$$

$$\approx \frac{1}{k}\left(\frac{f(x+h\hat{x}_i+k\hat{x}_j)-f(x+k\hat{x}_j)}{h} - \frac{f(x+h\hat{x}_i)-f(x)}{h} \right)$$

$$= \frac{f(x+h\hat{x}_i+k\hat{x}_j) - f(x+k\hat{x}_j) - f(x+h\hat{x}_i) - f(x)}{hk}$$

This estimate requires $\frac{1}{2}n(n+1)+n = \frac{1}{2}n(n+3)$ extra function evaluations. How should h and k be chosen? In this case, a similar analysis to the previous yields the compromise

$$h = k = (\text{eps})^{1/3}.$$

For double precision arithmetic, $h = k \approx 10^{-5}$.

— EXAMPLE CODE —

```
[f,g] = coolobjfn(x,params)
    n=length(x)
    f=
    if nargout > 1
        e=sqrt(eps)
        g=zeros(n,1)
        for k=1:n
            y=x
            y(k)=y(k)+e
            fe=coolobjfn(y,params)
            g(k)=(fe-f)/e
        end
    end
return
```

```
def coolobjfn(x,params,gflag)
    n=len(x)
    f=
    if gflag:
        e=np.sqrt(np.finfo(np.float64).eps)
        df=np.zeros((n,1))
        for k in range(n)
            y=x.copy()
            y[k] += e
            df[k]=coolobjfn(y,params,False)
        g=(df-f)/e
        return f,g
    else:
        return f
```